DAPC · SESSION 7

Low-risk releases

Ship a new version without fear, and go back in minutes when something is wrong.

AGENDA

Seven blocks, 205 minutes

Opening: the work trail from session 6	15 min
Theory with Claude, in three teams	45 min
Break	10 min
Hands-on: deploy, canary and rollback	85 min
Break	10 min
Skill: deploy from the runbook	20 min
Closing	20 min

Trainer explains · You practice · Break

OPENING

What the session 6 work trail shows

Test first

A failing test comes before the fix, and the pull request shows both.

Green CI

The “Unit tests” check is green before anyone merges.

A second pair of eyes

A classmate reviewed the change and left at least one comment.

Pick two pull requests from different people. Do they show all three?

WHY RELEASES ARE RISKY

Knight Capital: one old server

8
servers to update by hand

1
server missed the new code

45
minutes before it was stopped

$440 M
lost

The deployment was manual and it was the same everywhere only on paper. No one could go back fast. Today we fix both parts.

THEORY WITH CLAUDE

Three teams, three topics

TEAM A · pp. 129–134, 138–141

Risky releases and self-service deploys

Why releases hurt. Push-button deploys. Deployment is not release.

TEAM B · pp. 143–150

Environment-based patterns

Blue-green, canary, and the cluster immune system.

TEAM C · pp. 151–153

Application-based patterns

Feature toggles and dark launches.

Each team: 1. Ask Claude to explain the topic. 2. Check the answer against the official pages. 3. Show one example from the Summit Cafe API. Five minutes to teach the others.

KEY IDEA

Deployment is not release

Deployment

Install a version in an environment. Users may not see it yet.

A technical step. Our team controls it.

Release

Offer a feature to all customers, or to a part of them.

A business step. We can make it small and reversible.

Two ways to lower risk: change **where** the code runs (environment-based), or change **who sees** the feature (application-based).

SELF-SERVICE DEPLOYS

One button, the same way everywhere

AUTOMATIC

Unit tests

AUTOMATIC

Build image

AUTOMATIC

Publish to GHCR

ONE COMMAND

```
deploy-to-environment.sh ENV VERSION
```

- The same package goes to every environment.
- Every deploy records the command, who ran it and the output.
- A smoke test checks the app right after the deploy.

GHCR = GitHub Container Registry, GitHub's storage for Docker images.

VERSIONS

Image tags: a name for each version

```
ghcr.io/…/summit-cafe-api:1.2
```

A name people can say

1.1 and 1.2 are easy to say, to write in a runbook and to deploy.

A name that shows the code

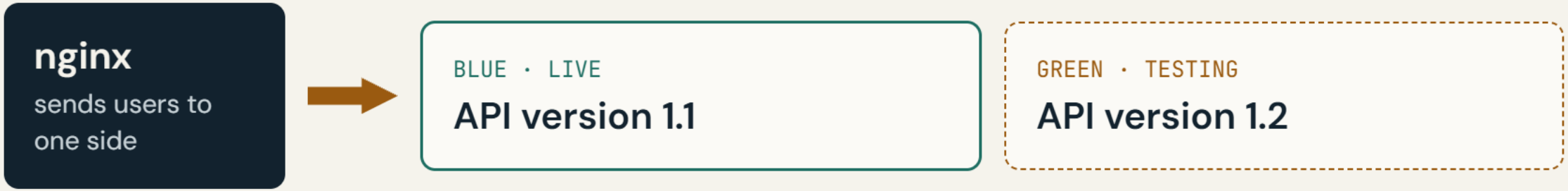
The tag sha-… points to the exact commit inside the image.

A name that never changes

1.1 is always the same image. That is why going back to 1.1 is safe.

Avoid “latest”. It points to a different image each time, so you cannot be sure what is running.

Blue-green: two copies, one switch



1. Deploy 1.2 to green. Nobody uses it yet.
2. Run the smoke test on green.
3. Switch nginx to green. Rollback means switching back to blue.

nginx = a web server that receives requests and passes them to the right backend.

WATCH OUT · DATABASES

Database changes in small steps

STEP 1 · EXPAND

Add the new column

Old and new code both work with the database.

STEP 2 · MOVE

Deploy the new code

It uses the new column. We can still go back.

STEP 3 · CONTRACT

Remove the old column

Only when the old code is gone for good.

Rule: never change the database in a way that breaks the version you may need to go back to.

ENVIRONMENT-BASED · PATTERN 2

Canary: a small group goes first

STEP 1

10%

9 of 10 requests stay on 1.1. One goes to 1.2. We watch errors and response time.

STEP 2

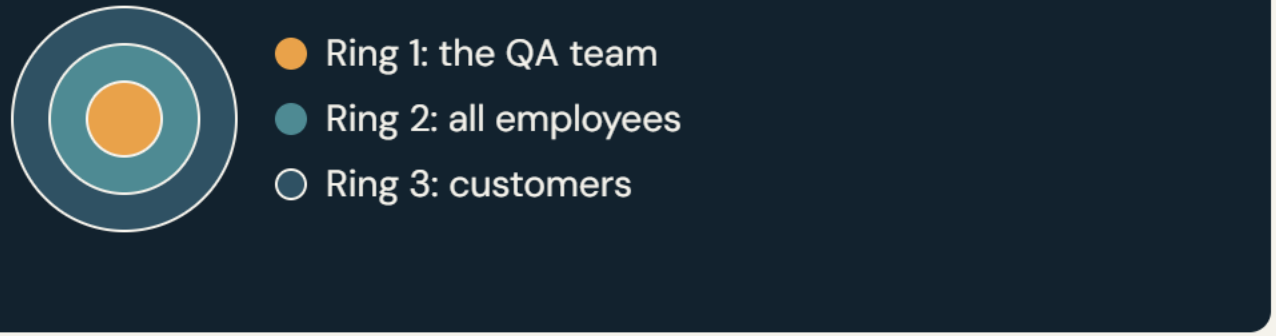
50%

Errors stay low, so half of the requests go to 1.2. Going back is still one change.

STEP 3

100%

All requests go to 1.2. Keep 1.1 ready for a few days, then retire it.



Immune system

1. Release 1.2 to a small group.

2. Monitoring watches the errors all the time.

3. Errors go up: traffic goes back to 1.1 by itself, and the team gets an alert.

APPLICATION-BASED

Feature toggles: a switch in the code

STEP 1 · DEPLOY

Version 1.2 goes live with the new feature turned off. Users see no change.

```
SCHEDULED_ORDERS=off
```



STEP 2 · RELEASE

We change the setting and restart. Users get the feature. No new deploy.

```
SCHEDULED_ORDERS=on
```

- To go back, turn the switch off. You do not need a new deploy.
- If the feature fails, the app should keep working with fewer features.
- Delete old toggles you no longer need. Too many switches make the code hard to read.

APPLICATION-BASED

Dark launch: run it, but hide it

1 · CUSTOMER SEES

The old result

The price of the order is calculated with the old method.

2 · HIDDEN

The new method runs too

The same order is calculated again in the background. Nobody sees it.

3 · WE COMPARE

Is the new one good?

Same price? Fast enough? Does it crash on some orders?

- Real traffic finds problems that tests do not find.
- Risk for users is almost zero: they only see the old result.
- Best for calculations and reads. Never for actions like charging a card.

SUMMARY

Which pattern for which risk

Pattern	Lowers the risk of	Rollback
Blue-green	A broken new version	Switch back to blue
Canary	Hurting all users at once	Send 0% to the new one
Feature toggle	A bad feature in a good version	Turn the switch off
Dark launch	Surprises under real load	Stop the hidden work

They work together: deploy often, release small, and always know the way back.

BREAK

Coffee time

Stand up, drink something, and come back fresh.



PRACTICE SETUP

Set up your computer

1. Docker Desktop

Install it and open it. Wait until it says “Engine running”.

```
docker ps
```

2. A bash terminal

Windows: Git Bash (it comes with Git for Windows). Mac and Linux: Terminal. Do not use CMD.

3. Get the code

Clone the course repository and go into the folder.

```
git clone https://github.com/devopscourseeng/devops-expert-dapc.git
cd devops-expert-dapc
```

4. Check that it works

Start the practice environment. You should see the menu.

```
./deploy/start-environment.sh 1.1
curl http://localhost:8200/menu
```

The image is public: no login is needed to pull it. Any token in the exercises is fake.

PRACTICE ENVIRONMENT

Your environments and commands

Where things run

Environment	Container	Port
Staging	cafe-staging	8100
Prod stable	cafe-prod-stable	8201
Prod canary	cafe-prod-canary	8202
Prod proxy (nginx)	cafe-prod	8200

In prod, always call port 8200. The proxy splits the calls.

Commands

```
# start prod with 1.1
./deploy/start-environment.sh 1.1
# deploy 1.2
./deploy/deploy-to-environment.sh staging 1.2
./deploy/deploy-to-environment.sh prod 1.2
# send 10% of the calls to 1.2
./deploy/set-weight.sh 10
# which version answered?
curl http://localhost:8200/version
```

Deploying to prod does not release: the canary gets 0% of the calls until you change the weight. Weight 0 is the rollback.

PRACTICE · FEATURE TOGGLE

Try the feature toggle: off, then on

A toggle is a setting in the code. You change the setting and restart; you do not change the code. Do the four steps in order.

1 **Deploy 1.2 with the toggle off**
Then check: it says “off”.

```
./deploy/deploy-to-environment.sh staging 1.2 --scheduled-orders off
curl http://localhost:8100/version
```

2 **Make an order with a pickup time**
The answer has no pickup_time. The toggle is off.

```
curl -X POST http://localhost:8100/orders -H "Content-Type:
application/json" -d '{"items":
[{"id":1,"quantity":1}],"pickup_time":"14:30"}'
```

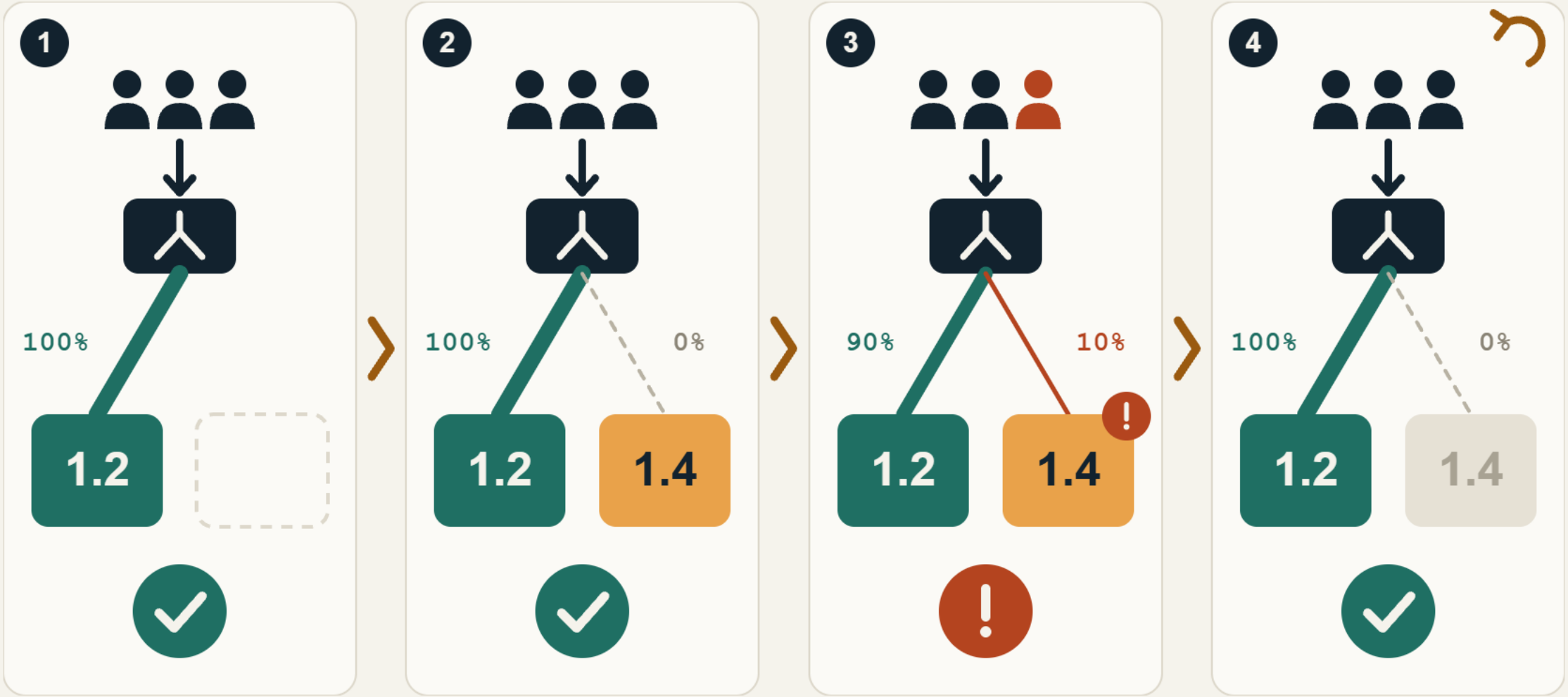
3 **Deploy 1.2 with the toggle on**
Same version, one word changes. Check: it says “on”.

```
./deploy/deploy-to-environment.sh staging 1.2 --scheduled-orders on
curl http://localhost:8100/version
```

4 **Make the same order again**
Now the answer shows "pickup_time":"14:30".

```
(the same curl command as in step 2)
```

Finished? Do the exercises on the next slide, if you have not done them yet.


1
100%
1.2
2
100%
0%
1.2
1.4
3
90%
10%
1.2
1.4
4
100%
0%
1.2
1.4

HANDS-ON · 85 MINUTES

Deploy, canary and rollback with our scripts

Version 1.2 is good. Version 1.4 is broken. Find the commands yourself: deploy/ scripts, the cheat sheet and the README. Do the steps in order.

Start with 1.2 as the base Prod (stable) and staging both run the good version 1.2.	YOU CAN SHOW Which version answers on port 8200 and on port 8100.
Try the broken 1.4 in staging Is it safe to release?	YOU CAN SHOW Your answer, yes or no, and the proof.
Deploy 1.4 to prod as a canary Customers do not use it yet. Does prod still work?	YOU CAN SHOW Prod still answers with 1.2, and the proof.
Send 10% of the calls to 1.4 What happens to prod now?	YOU CAN SHOW Errors that come and go, found by your check.
Roll back to 1.2, and time it Customers must stop seeing errors.	YOU CAN SHOW How many seconds it took, and proof that all calls work.

BREAK

Stretch and drink water

Move a little and rest your eyes. We continue soon.



RUNBOOK (DESPLIEGUE)

#	Step	Command	How we know it is OK	How we go back
1	Check ready	curl localhost:8200/menu	Menu answers	Stop here
2	Test in staging	./deploy/deploy-to-environment.sh staging 1.4	Prints OK. Staging answers on 8100	Stop here
3	Deploy to prod	./deploy/deploy-to-environment.sh prod 1.4	Prints OK. Canary answers on 8202	./deploy/set-weight.sh 0
4	Weight 10	./deploy/set-weight.sh 10	Prints OK	./deploy/set-weight.sh 0
5	Watch	./deploy/check-menu.sh	Prints OK: all calls are 200	./deploy/set-weight.sh 0
6	Weight 50	./deploy/set-weight.sh 50	Prints OK	./deploy/set-weight.sh 0
7	Weight 100	./deploy/set-weight.sh 100	Prints OK	./deploy/set-weight.sh 0

BIG IDEA · HARNESS ENGINEERING

Build a safe place for the agent to work

Without a harness

```
docker pull ghcr.io/…/api:1.2
docker run -p 8202:8000 …
edit nginx.conf, reload nginx
curl localhost:8200/menu
```

With a harness

```
./deploy/deploy-to-environment.sh
prod 1.2
```

One safe button. The same way every time.

Rules
The runbook says what to do.

Brakes
Checks catch mistakes.

Safe button
deploy-to-environment.sh works in one way only.

Approval
A person says yes to a rollback.

Do not try to make the agent perfect. Make its mistakes small, easy to see and easy to undo.

CLAUDE SKILL

Skill: deploy from the runbook

Reads

The runbook in the repo: the context first, then one step per row.

Does

Deploys, sets the weight, runs the check after each step, and shows the result.

Suggests

A check fails: it proposes the rollback and waits for your yes.

- It runs only commands that are written in the runbook.
- If a command or a check is missing, it asks. It never guesses.

CLOSING

What you can do now

- Explain the difference between deployment and release.
- Deploy one version with one command.
- Choose blue-green, canary or a toggle for a given risk.
- Go back to the last good version, and prove how long it takes.

NEXT · SESSION 8

Low-risk architecture and the end of the First Way.